

Running ChatSaints locally

Everything here is free. No mail provider, no cloud storage, no hosting, no accounts to sign up for. The whole stack runs on your machine.

Works on **Windows, macOS and Linux**. Windows differences are called out where they exist.

Quick start

If you already have Bun, Docker and Flutter:

```
git clone https://github.com/Emmanuelkelly29/chatsaints.git
cd chatsaints

docker compose -f infra/docker-compose.dev.yml up -d      # Postgres + Redis
cp apps/api/.env.example apps/api/.env                    # Windows: copy
apps\api\.env.example apps\api\.env

bun install
bun run db:migrate                                         # create the schema
bun run db:seed                                           # scriptures +
geography

bun run dev                                                # API on :4000
```

Then in a second terminal:

```
cd apps/mobile
flutter pub get
flutter run
```

The rest of this document explains each step and what to do when one fails.

1. Prerequisites

Tool	Version	Why
Bun	1.3+	Runs and installs the API. Replaces Node and npm here.
Docker Desktop	any current	Provides Postgres and Redis. Free for individuals and small companies.
Flutter	3.44+	Only needed for apps/mobile .

Tool	Version	Why
Android Studio	any current	Only needed to run the mobile app on an emulator.

Windows: install Bun with `powershell -c "irm bun.sh/install.ps1 | iex"`. Use **PowerShell**, not `cmd`. Docker Desktop needs WSL2, which its installer sets up.

You do **not** need Postgres or Redis installed natively. Docker provides both.

If you would rather not use Docker

Install PostgreSQL 16+ and Redis 7+ yourself, create a database, and point `DATABASE_URL` and `REDIS_URL` at them. Everything else is identical. Redis on Windows is awkward without Docker or WSL, which is the main reason Docker is the recommended path.

2. Get the code

```
git clone https://github.com/Emmanuelkelly29/chatsaints.git
cd chatsaints
```

Enable the pre-commit hook once per clone. It blocks committing a `.env` file or a hardcoded credential:

```
git config core.hooksPath .githooks
```

Windows: the hook is a shell script, so it needs **Git Bash**, which ships with Git for Windows. It runs automatically when you commit from any client.

3. Start the database and cache

```
docker compose -f infra/docker-compose.dev.yml up -d
```

That gives you PostgreSQL 18 on `localhost:5432` and Redis 7 on `localhost:6379`, with data kept in Docker volumes across restarts.

Check they are healthy:

```
docker compose -f infra/docker-compose.dev.yml ps
```

Already running Postgres on 5432? Change the left-hand port in `infra/docker-compose.dev.yml` (for example `"5433:5432"`) and update `DATABASE_URL` to match, rather than stopping your existing

server.

Useful later:

```
docker compose -f infra/docker-compose.dev.yml down      # stop
docker compose -f infra/docker-compose.dev.yml down -v   # stop and DELETE
all data
```

4. Configure the API

```
cp apps/api/.env.example apps/api/.env
```

Windows: `copy apps\api\.env.example apps\api\.env`

The defaults already match the Docker setup, so there is **one** value to fill in. Generate a JWT secret:

```
openssl rand -hex 64
```

Windows (PowerShell), if you do not have `openssl`:

```
-join ((1..64) | ForEach-Object { '{0:x2}' -f (Get-Random -Max 256) })
```

Paste it into `JWT_SECRET=` in `apps/api/.env`.

The server refuses to start if `JWT_SECRET` is missing or shorter than 32 characters, and tells you exactly which variable is wrong. That is deliberate: a misconfigured server should fail at boot, not on the first login.

Leave everything else alone. In particular **leave all `SMTP_*` empty**, which is what makes email free. See step 7.

5. Create the schema and load reference data

```
bun install
bun run db:migrate
bun run db:seed
```

- `db:migrate` applies the migration and regenerates the Prisma client.
- `db:seed` loads 191 scripture verses and the church geography (areas, coordinating councils, stakes, missions). It is safe to run repeatedly.

6. Run the API

```
bun run dev
```

You should see:

```
{"level":"info","message":"websocket server ready","path":"/ws"}
{"level":"info","message":"postgres connected"}
{"level":"info","message":"redis connected"}
{"level":"info","message":"api listening","port":4000,...}
```

Confirm it: <http://localhost:4000/health>

It reloads automatically when you change a file. **Keep this terminal visible**, because verification codes are printed here.

7. Signing in, and where the code goes

There is no mail provider in development. When `SMTP_*` is empty and `NODE_ENV` is not `production`, the API prints the email to the terminal instead of sending it:

```
DEV EMAIL – not sent, SMTP is unconfigured
To:      ada@example.org
Subject:  ChatSaints: Verify your email to finish signing up

CODE:    392518

Configure SMTP_* in apps/api/.env to send real mail.
```

So the flow is: register in the app, look at the API terminal, type the code in. Same for signing in with a code and for the monthly session refresh.

The code is **never** returned in an HTTP response, in any environment. It is only ever printed to the terminal of the machine running the server, and only outside production. In production a missing SMTP configuration is a hard 503, not a fallback.

8. Run the mobile app

```
cd apps/mobile
flutter pub get
```

```
flutter run
```

Or from the repository root: `bun run mobile:dev`

The API must already be running. The client finds it automatically:

Where the app runs	API address used
Android emulator	<code>http://10.0.2.2:4000</code>
iOS simulator, desktop	<code>http://localhost:4000</code>
Physical device	you must pass it explicitly

A physical phone is not your computer, so it needs your machine's LAN address:

```
flutter run --dart-define=API_BASE_URL=http://192.168.1.10:4000
```

Find your IP with `ipconfig` on Windows or `ifconfig | grep inet` on macOS. Both devices must be on the same network.

Everyday commands

All from the repository root.

Command	Does
<code>bun run dev</code>	Start the API with reload
<code>bun run check</code>	Typecheck, lint and test in one pass
<code>bun run test</code>	Tests only
<code>bun run db:studio</code>	Browse the database in a GUI
<code>bun run mobile:dev</code>	Run the Flutter app
<code>bun run mobile:analyze</code>	Static analysis on the Flutter code

Run `bun run check` before you push. CI runs the same three gates and will fail the build otherwise.

Changing the database

`apps/api/prisma/schema.prisma` is the single source of truth. **Never hand-write SQL and never edit a file under `prisma/migrations/`.**

```
# 1. edit apps/api/prisma/schema.prisma
# 2.
```

```
bun run db:migrate
```

Prisma compares your schema against a temporary shadow database, writes the SQL migration for you, applies it, and regenerates the client. You should never need to write a migration by hand.

Command	Does
<code>bun run db:migrate</code>	Create and apply a migration, then regenerate the client
<code>bun run db:deploy</code>	Apply existing migrations without creating one (production)
<code>bun run db:generate</code>	Regenerate the Prisma client only
<code>bun run db:reset</code>	Drop, recreate and re-seed. Deletes all local data.

Commit the generated folder under `prisma/migrations/` along with your schema change.

Media and uploads

Uploads go to `apps/api/uploads/` on disk. Nothing is sent anywhere and nothing costs money.

Images are processed once at upload:

	Longest edge	Quality
Chat media	1920px	80
Profile photo (<code>?purpose=avatar</code>)	512px	80

A 4032x3024 phone photo becomes 1920x1440 as chat media, or 512x384 as an avatar. All metadata is stripped, which also removes the **GPS coordinates** phone cameras embed in photos.

Tune it in `.env` with `MEDIA_IMAGE_MAX_DIMENSION`, `MEDIA_IMAGE_QUALITY`, `MEDIA_AVATAR_MAX_DIMENSION`, `MEDIA_AVATAR_QUALITY` and `MEDIA_MAX_UPLOAD_MB`.

Files are **not** served from a public path. Every download goes through `GET /api/media/file/:ownerId/:fileName`, which checks the caller is allowed to see the owning message or status and returns 404 if not.

Switching to cloud storage later means setting `STORAGE_TYPE=s3` and implementing the S3 driver. No calling code changes.

What costs money, and when

Concern	In development	In production
Database, cache	Docker containers, free	Managed Postgres and Redis
Email	Printed to terminal, free	Real SMTP credentials required
File storage	Local disk, free	Object storage via <code>STORAGE_TYPE=s3</code>

Concern	In development	In production
Push notifications	Not configured, no-op	Firebase service account
Video calls	Reports unavailable	LiveKit credentials
Device management	Reports unavailable	MaaS360 credentials

Anything unconfigured reports itself as unavailable rather than pretending to work. That is deliberate: an earlier version of this backend returned `{success: true}` for device enrollment that never happened, and issued fake video call tokens.

Troubleshooting

Invalid environment configuration The server tells you which variable is wrong. Usually `JWT_SECRET` is empty or too short. See step 4.

postgres connected never appears / ECONNREFUSED The containers are not up. Run `docker compose -f infra/docker-compose.dev.yml ps`. On Windows, check Docker Desktop is actually running.

Port 4000 already in use Something else is on it, often an older API instance. macOS or Linux: `lsof -ti:4000 | xargs kill`. Windows: `netstat -ano | findstr :4000` then `taskkill /PID <pid> /F`.

Registration returns 503 Only happens if `NODE_ENV=production` in your `.env`. It should be `development` locally.

No verification code arrives It is not emailed. Look at the terminal running `bun run dev`. See step 7.

The app cannot reach the API On an emulator this is almost always the address. Android emulators reach the host at `10.0.2.2`, not `localhost`. On a physical device you must pass `--dart-define=API_BASE_URL=http://<your-lan-ip>:4000`.

flutter clean hangs Known to hang on some machines at 0% CPU without deleting anything. Delete the directories directly instead:

```
rm -rf build android/.gradle android/build .dart_tool
```

Windows PowerShell:

```
Remove-Item -Recurse -Force build, android\.gradle, android\build, .dart_tool
```

Gradle warns about file_picker and flutter_webrtc Expected. Both still apply the Kotlin Gradle Plugin, which Android Gradle Plugin 9 has moved away from. They are warnings, not errors, and the build proceeds.

The first Android build takes a very long time It is downloading Gradle dependencies and merging native libraries, and `build/` will reach a few gigabytes. Later builds are much faster.

iOS: only Chrome is offered as a device Flutter does not support Xcode 27 / iOS 27 yet. Tracked upstream in flutter/flutter [#187743](#) and [#187781](#). Use an Android emulator until it lands.

A commit is rejected mentioning a hardcoded credential The pre-commit hook found something credential-shaped. Move the value into `apps/api/.env` (which is gitignored) and read it through `src/config/env.ts`. If it is genuinely a false positive, widen the allowlist in `.github/hooks/pre-commit` rather than bypassing the hook.

Project layout

<code>apps/api/</code>	TypeScript API. Express 5, Prisma 7, PostgreSQL, Redis, WebSocket
<code>apps/mobile/</code>	Flutter client (Android, iOS, web)
<code>infra/</code>	nginx config and the development docker-compose file
<code>docs/</code>	this document
<code>.github/</code>	CI
<code>.githubhooks/</code>	pre-commit secret scanning

Inside `apps/api/src`:

<code>config/</code>	validated environment, Redis
<code>domain/</code>	role hierarchy and permission tiers
<code>features/</code>	one folder per area: <code>routes.ts</code> , <code>service.ts</code> , <code>schemas.ts</code>
<code>lib/</code>	Prisma client, logger, tokens, normalization, serialization
<code>middleware/</code>	auth, validation, error handling
<code>ws/</code>	WebSocket server and handlers

Responses are snake_case on the wire and camelCase in TypeScript. The conversion happens once in `middleware/serializeResponse.ts`, so handlers never think about it.